Addestramento supervisionato → Machine learning

$D_{train} = \{ (\underline{x}, \underline{y}) : \underline{x} \in \mathbb{R}^N, \ \underline{y} \in \mathbb{R}^N \}$

$D_{val} \cap D_{train} = \emptyset$

$D_{test} \cap D_{train} = \emptyset$

Loss: MSE → Penalizza di più gli errori > 1

$\mathcal{L} = \| F(\underline{x}) - \underline{f}(\underline{x}) \|^2$ (output modello, y dataset)

Geometria del bravo → L1 $\mathcal{L} = \| F(\underline{x}) - \underline{f}(\underline{x}) \|$ Calcola lo spostamento lungo gli assi

Fase di training su D_{train}

- Minimizzare la loss → discesa del gradiente
- Modificare pesi e bias della rete
- $\underline{\theta}_k = \underline{\theta}_{k-1} - \eta \nabla \mathcal{L}(\underline{\theta}_{k-1})$
- $\theta = (W, b)$

Finito il training si testa il modello su D_{test}

- Modello pronto → Fase di inferenza

Algoritmo di training

- Questo per ogni stato della rete
- Inizializzazione dei pesi e bias
- Passo forward, si calcola l'output
- Passo backward, propagazione dell'errore all'indietro

Parametri:

- η: learning rate
- N_{epoche}: num. epoche
- D_{train}
- Architettura rete: L (stati nascosti)

Aggiornamento pesi e bias
(A fine iterazione di training)

Eq. back propagation:

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(l)}} = \frac{1}{n} \sum_{k=1}^{n} \delta_j^{(l)} \cdot \sigma(s_i^{(l-1)})$$

$$\frac{\partial \mathcal{L}}{\partial b_j^{(l)}} = \frac{1}{n} \sum_{k=1}^{n} \delta_j^{(l)}$$

} Calcolano i gradienti della loss

Modo efficiente per calcolare $\nabla \mathcal{L}$, grazie alla chain-rule possiamo calcolare $\frac{\partial \mathcal{L}}{\partial w_{ij}}$, $\frac{\partial \mathcal{L}}{\partial b_j}$